

doTrees() review: what was fixed, changed and deleted

Adversarial multi-agent review of postestimate_doTrees.R, helper_doTrees.R and their tests — branch igascaTrees

1 Scope and method

Three independent reviewers were run against R/postestimate_doTrees.R and R/helper_doTrees.R: two bug hunters given disjoint angles (control flow and accumulator state; statistical correctness and cross-function API contracts) and one documentation-concision pass over those two files plus tests/testthat/test-postestimate_doTrees.R and tests/testthat/test-helper_doTrees.R. Neither bug hunter saw the other's findings. Every claim below was then re-verified independently before being acted on; three reported findings were discarded at that stage, and two were found by the verification pass rather than by any reviewer.

Verification used partykit 1.3.0 and the shipped fixture tests/testthat/data/igscaTrees.Rdata (n = 1000).

Result of the full suite after all changes: 791 passing, 0 failures, 0 errors, 0 warnings, including the snapshot test (which is unchanged) and the verify file that B4 touches.

One finding was applied and then withdrawn. B5 proposed reinstating max_cuts; it was reverted on the author's challenge once the git history showed the removal had been deliberate. The numbering below skips B5 deliberately, and section 4.4 records what happened and why.

2 Correctness fixes

2.1 bug B1 — root_criteria() documented a contract it does not honour
Where. root_criteria() roxygen, R/helper_doTrees.R.

What was wrong. The documentation stated that NULL criteria occur “only when no test ran at all, which means the root trafo failed and n_fail_full is 1”. partykit:::.extree_node() returns a bare partynode carrying no info on **four** paths, only one of which sets n_fail_full:

Path in .extree_node()	n_fail_full
is.null(thismodel) — root trafo failed	1
depth >= ctrl\$maxdepth	0
sw < ctrl\$minsplit	0
all(is.na(crit)) — e.g. every covariate constant	0

Measured on the fixture:

```
maxdepth = 0      n_fail_full = 0    root_criteria NULL = TRUE
minsplit = 1010   n_fail_full = 0    root_criteria NULL = TRUE
```

A caller separating “the root failed” from “the root was tested and stopped” by is.null(root_criteria(tr)) therefore gets the wrong answer. The existing test “root_criteria survives a tree that never splits” asserts the **inverse** invariant but never exercises these three routes.

Fix. The roxygen now enumerates all four paths and states explicitly that NULL on its own does not imply a failure. Behaviour is unchanged — this was a documentation defect about real behaviour, not a code defect. A new test, “root_criteria() is NULL whenever no test ran, not only on failure”, pins the two reachable non-failure routes.

2.2 bug B2 — every failed node fit was counted twice and refitted

Where. the ytrafo failure branch in `doTrees()`; `attach_leaf_fits()` and `new_collector()` in `R/helper_doTrees.R`.

What was wrong. When the trafo fails, `partykit:::extree_fit()`'s `updatetrafo()` wrapper returns `NULL` (line 116–117 of its body), so `.extree_node()` returns `partynode(id)` **with no info at all**. That makes a node whose fit failed indistinguishable from a leaf the trafo never visited. `attach_leaf_fits()` consequently refitted it — the identical rows with the identical arguments, so a deterministic repeat of the same failure — and incremented `n_fail_leaf` on top of the `n_fail_full` / `n_fail_node` already booked.

Measured, with `try_fit()` mocked to always fail:

```
try_fit() calls: 2    (on the identical full sample)
n_fail_full = 1    n_fail_node = 0    n_fail_leaf = 1    <- one event, two counters
```

This is exactly what the comment at `test-postestimate_doTrees.R:463` says must not happen; the existing test covered only the success path.

Fix. `new_collector()` gained a `failed_subsets` list. The trafo records the row set via a new `record_failed_subset()` whenever the node fit fails, and `attach_leaf_fits()` consults `subset_failed()` before refitting, returning the `converged = FALSE` stub directly. This removes both the double count and one guaranteed-to-fail IGSCA fit per failed node. New test: "a root fit failure is booked once, not again as a leaf failure" (asserts `try_fit()` is called exactly once and `n_fail_leaf == 0`).

Related comment corrected. `n_fail_node`'s inline description claimed it "includes inner nodes". A node whose trafo failed can never become an inner node, so that state is unreachable; the comment now says so.

2.3 bug B3 — split counters were per node, not per covariate

Where. `argmax_split()`, `warn_dead_splitter()`, and three comments.

What was wrong. `scanned` and `got_stat` were single flags over the whole `whichvar` loop, and the counters were incremented **after** it. So a node that scanned two covariates incremented `n_split_scan` by one. Since `warn_dead_splitter()` fires only when `n_fail_split == n_split_scan`, a kernel that dies on every factor covariate but works on numerics reported **zero** failures and produced **no warning**, while silently biasing variable selection toward the covariates it happens to survive.

All three reviewers reached this independently. Three comments documented the per-covariate reading the code did not implement (`# Successful scan of covariate`, `# Failure to split on this covariate`, and `# Number of scans in a selected covariate on the igsca_info attribute`).

Fix. The flags are gone; both counters are incremented inside the loop, once per covariate the kernel was actually invoked on. `warn_dead_splitter()`'s message now says "covariate scan(s)" rather than "node(s)". The existing dead-kernel test still holds (a kernel that throws on everything fails every scan, so the two counters remain equal).

2.4 bug B4 — `verify()` check 6 inverted its quantifier

Where. `R/postestimate_verify.R`. Outside the two files under review, but it is the guard `csem_converged()` — and therefore `try_fit()`\$ok — rests on, so it was in scope by consequence.

What was wrong. `stat["6"] <- !any(zeroness)`. `zeroness` holds one element per constraint, `TRUE` meaning "honoured", so a problem is **any** element being `FALSE` and the test should be `!all()`.

With `!any()` the check fired only when all five constraints failed simultaneously. Worse, for plain GSCA `zerness["UniqueScore"]` and `zerness["UniqueLoading"]` are structurally TRUE, so `any(zerness)` was **always** TRUE and check 6 could never fire at all.

```
zerness = c(Structural = FALSE, Measurement = TRUE, Weight = TRUE,
            UniqueScore = TRUE, UniqueLoading = TRUE)
!any(zerness) = FALSE    -> no problem reported    (old)
!all(zerness) = TRUE     -> problem reported       (new)
```

A node or candidate fit with a non-zero path estimate on a structural zero passed `sum(verify(fit)) == 0`, so `csem_converged()` called it admissible and it could win the argmax.

Fix. `!any()` → `!all()`, with a comment recording why. The `verify` test file and the whole suite pass unchanged.

2.5 bug B6 — the FITdiff kernel lacked the guard its siblings have

Where. `partition_stat()`, FITdiff branch.

What was wrong. The distance branch checks `!is.finite(val)` and increments `n_fail_candidate`; the FITdiff branch returned `val` unchecked. `calculateFIT` is `1 - SS_unexplained/SS_total`, so a degenerate node yields NaN/Inf with no error. NaN is argmax-safe, but Inf is not: `any(is.finite(c(Inf, 0.3)))` is TRUE and `which.max()` selects it, so a meaningless candidate becomes the split. Either value also left `n_fail_candidate` at 0 where the DGi/DLi path would have incremented it, making the counter incomparable across splitters.

Fix. The branch now mirrors the distance branch. Verified: an Inf FITdiff returns `NA_real_` and increments `n_fail_candidate`.

2.6 bug B7 — coef()’s drop collapsed more than it documents

Where. `coef.igsca_tree()`.

What was wrong. `@param drop` promises that “a single-node result is simplified from a one-row matrix to a named vector”, but the implementation was `drop(cf)`, which also collapses a single-column result. A multi-leaf tree whose leaves all failed has `nobs` as its only column, so `coef()` returned a bare named vector and lost the row-per-node contract stated in `@returns`.

Fix. `if (drop && nrow(cf) == 1L) cf[1L, ] else cf`. New test “`coef()` keeps its matrix shape when every leaf refit failed” builds the case by failing every fit below the root.

2.7 bug B8 — the 11-level refusal fired from the wrong place

Where. `validate_tree_input()`.

What was wrong. `validate_tree_input()` takes a `splitter` argument and never read it, so the one splitter-specific precondition — unordered factors of at most 11 levels — was enforced only from inside `candidate_partitions()`. That runs inside `partykit`’s `splitfun`, **outside** `argmax_split()`’s `tryCatch`, so it aborted `ctree()` after an arbitrary amount of tree growth and named column 15, the model-frame column index (indicators occupy columns 1..J), rather than the user’s covariate.

Fix. The same limit is now checked up front, by covariate name, and only for the non-native splitters (the native scan does not refit, so it is exempt). The defensive check inside `candidate_partitions()` stays for direct calls. New test: “`validate_tree_input()` refuses an unaffordable factor by name, up front”.

3 Documentation and comments

3.1 Comments that were factually wrong

Location	What was wrong
<code>attach_leaf_fits()</code> , the <code>info[names(...)]</code> <- ... line	Said “Rewrites the entire info”. It merges , and the merge is precisely what the “root_criteria survives a tree that never splits” test exists to protect. The comment described the regression rather than the fix.
<code>argmax_split()</code> , multiway branch	“Placeholder for multiway_split functionality (not currently supported)”. <code>multiway_split()</code> is fully implemented and has a passing test; it is merely unreachable from doTrees() . Now says so (and the typo is gone).
<code>doTrees()</code> @details	“maximize the difference in either FIT, DLi or DGi”. Only FIT is a difference; DLi/DGi are matrix distances and are never subtracted.
<code>partition_stat()</code> roxygen	Titleless (leading blank #'); “(NPT)” is vestigial vocabulary appearing exactly once in all of R/; and “DGi”/“DLi” only is contradicted by the function’s own <code>stat_kinds</code> , which accepts DGc/DLc too.
<code>drop_inner_node_objects()</code> roxygen	“see the note in [doTrees()]” — no such note exists, and the only related text there points back here. Replaced with the actual mechanism (<code>saveinfo = TRUE</code>).
<code>candidate_partitions()</code> , # observed levels	Sat on <code>levels(z)</code> , which is the covariate’s full level set including levels absent from the node — the exact distinction the next line and the “enumerates the node’s levels” test turn on.
<code>argmax_split()</code> roxygen	Described it as maximizing a pooled-vs-MGA difference. It maximizes whatever <code>splitter</code> returns; the tests hand it arbitrary kernels.
<code>igsca_info</code> counter comments	See B2 and B3.

3.2 Deleted outright

postestimate_doTrees.R: # Preparation; # The main workhorse; # Safely fetches `.object$Information$Arguments` (the callee’s roxygen says it); # from `.influence` character string and # From `.splitter` (the same note twice, both restating a `match.arg` a hundred lines earlier); # Bonferonni corrected p-values (restates the string literal, misspelled); the # `testtype` is null if the above don't fire note (restates `is.null`); the “Exact matching, no regex” sentence; and the stray # See `partykit::ctree_control()`\$`splitfun` on the formula, which pointed at the wrong consumer.

helper_doTrees.R: # Equivalent to `uz[1:(length(uz)-1)]`; # Create list of break points... and the abandoned # Given; # `boomer::boom(...)`; # This is the idiom to go (unfinished); # Here is where the maximum is taken; the duplicated # looping over `whichvar` is the same idiom (kept at the loop, dropped from the assignment); #' @returns List on `new_collector()` (it has no man

page and no `NAMESPACE` entry, so a titleless block generated no `.Rd` at all — and it is an environment, not a list); and the whitespace-only `@description` in `influence_vec()`.

Tests: `# TODO: ?`, `# TODO: Why should these be identical?` (replaced by the answer: the mock throws on every candidate), `# FIXME: This might be overkill`, `consider deleting` (the `test` was kept — its own comment names a mutation that leaves every other test in the file green), `# print(calls$n)`, the fourteen commented-out “Vector of Residuals” blocks (they would fail if uncommented, since `match.arg` rejects “vec”), and two comments restating `match(..., names(...))`.

3.3 Condensed

The nine-line `# debug(partykit:::.split)` ladder became four lines keeping the vignette pointer and the call chain, which is the part that is painful to rediscover. The five-line `airquality` transcript in the `trafo` became two lines keeping the `unclass(...)$trafo` and `partykit:::.y2infl()` pointers. The six-line `plot(airct)` recipe above the numeric branch became one line, since its conclusion is already stated by `right = TRUE`’s own comment and by a test. Meta-commentary (“In case I have forgotten why subset is here”, “AFAIK, this is just used later for `prprinting`”) was rewritten as statements of fact, and the typos `prprinting`, `smaller that`, `funcitonality` and `is forces` were fixed.

`candidate_partitions()`’s 18-line `@examples` block was folded into a four-line description: the block sits in a `@noRd` function, so it was neither rendered by roxygen nor run by `R CMD check`, and each convention it showed is already pinned by an executing test.

3.4 Duplication collapsed

`warn_dead_splitter()`’s “a throwing kernel becomes NA becomes a clean-looking stump” explanation appeared three times (its own roxygen and both test files); the tests now cite `?warn_dead_splitter`. The `TREETEMPGROUP` rationale appeared three times; `doTrees()`’s copy is canonical because it explains a `stopifnot` a user can trip. The `.extree_node()` `minprob` formula appeared twice; `@param minprob` is canonical. “`ytrafo` is not called at terminal nodes” appeared three times; `attach_leaf_fits()`’s roxygen is canonical. A note about `z_true` offering a kernel no freedom appeared twice in one test file, 200 lines apart.

3.5 Filled in

`@param coin_distribution` documented neither of its two values; it now says what “approximate” and “asymptotic” actually do. `doTrees()`’s `@references` block contained `\insertAllCited{}` with no `\insertCite` anywhere in the block, so `man/doTrees.Rd` rendered an **empty** References section; the details paragraph now cites Hwang2021a (already in `inst/REFERENCES.bib`). `validate_tree_input()`’s roxygen restated, in full, a fact already stated inline at the check that enforces it.

Also: `(break)()` in `argmax_split()` became `break`. It behaves identically (verified) but reads as a typo.

4 Considered and deliberately not applied

4.1 **not applied** The **DLi/DGi** kernels are not sample-size normalized

Both are raw distances between a pooled matrix estimated on n rows and group matrices estimated on n_1, n_2 ; under the null their expectation behaves roughly as $c/n_1 + c/n_2$, which is minimised at $n_1 = n_2$ and diverges as either group shrinks. Maximising such a quantity biases the cutpoint toward the `minbucket` boundary. Measured on 500 rows drawn from a **single** true group and split on pure noise, **DLi** ran 0.296 / 0.122 / **0.091** / 0.211 / **0.344** across `n_left = 70 ... 430` — the `argmax` at the most extreme admissible cut, $3.8\times$ the minimum, with no heterogeneity present at

all. `FITdiff`, a variance-explained ratio pooled over all rows, was flat (± 0.003) over the same range.

Not applied, and confirmed as a deliberate decision: normalizing the statistic changes what the method estimates. That is a decision about the method, not a defect in its implementation. It is recorded here as the single most consequential open question the review raised.

4.2 **not applied** Missing covariate values are routed by an unrepeated coin flip

4.2.1 The mechanism

`partykit::kidids_node()` routes a row whose split variable is `NA` in one of two ways. If the node carries surrogate splits it uses those, deterministically. Otherwise:

```
prob <- prob_split(primary)
x[nax] <- sample(1:length(prob), sum(nax), prob = prob, replace = TRUE)
```

`doTrees()` sets `maxsurrogate = 0L`, so there are never surrogates and every `NA` row is assigned by an independent draw. That draw is made **twice on different occasions**: once inside `.extree_node()`, fixing the rows the node's model was fitted on, and again in `ctree()`'s closing `fitted_node()` call, fixing `tree$fitted`. Nothing ties the two together.

Measured on the fixture with 20% of `z_true` set to `NA`:

```
prob_split(root)                = c(0.50125, 0.49875)
two fitted_node() draws on one tree disagree on 106 of 1000 rows
```

`prob` is close to a fair coin, so roughly half of the 200 missing rows land elsewhere on every re-evaluation. Because `attach_leaf_fits()` reads `tree$fitted`, a leaf that needs a refit is fitted on the **fitted** partition while its siblings keep fits from the **growth** partition, and `predict()` may disagree with `coef()` on the same object.

4.2.2 The framing that matters

Only options 1–4 below remove the ambiguity. Options 5 and 6 merely choose **which** of the two partitions the stored fits agree with; the object still carries two, because the randomness is in `partykit`'s routing, not in where `doTrees()` reads it from. That is the argument for fixing this at the routing end rather than at the bookkeeping end.

4.2.3 The options

Option 1 — refuse NA covariates in `validate_tree_input()`. One `stop2()`, no interaction with `partykit` at all, and consistent with the package's existing stance: `processData()` already **errors** on missing indicators rather than dropping rows. The ambiguity cannot then arise. Cost: a behaviour change for anyone with partially observed covariates, who must impute or subset first — but that is a decision they should be making knowingly anyway. **This is the recommended default.**

Option 2 — majority = TRUE in the `ctree_control()` call. `.extree_node()` reacts to it by collapsing `prob` to an indicator:

```
prob <- tabulate(kidids) / length(kidids)
if (ctrl$majority) prob <- as.double((1L:length(prob)) %in% .which.max(prob))
```

`sample()` on a degenerate `prob` always returns the same kid, so routing becomes deterministic. Verified: the same tree that disagreed on 106 rows disagrees on **0** once `prob` is `c(1, 0)`. This is a one-line change and keeps every row in the analysis. Cost: it silently decides that all missing rows

belong to the larger kid, which is an imputation rule, not a neutral one — deterministic is not the same as correct. If taken, it should be exposed as an `igsca_tree_control()` argument rather than hard-coded, so the choice is visible in the fitted object.

Option 3 — surrogate splits (`maxsurrogate > 0`). Routes missing rows by a correlated covariate: deterministic **and** data-driven, and the only option here that uses information about the row rather than about the node. Requires undoing `cc$vsplitfun <- cc$splitfun` and restoring partykit's own `.ctree_split()`, because surrogate splits should be found by association rather than by IGSCA refits — as currently wired, each surrogate candidate would cost a two-group refit. Needs covariates that actually correlate, so it does nothing for a single-covariate tree.

Option 4 — MIA (`MIA = TRUE`). Treats NA as its own category and lets the split search decide which side it belongs on — statistically the most defensible of the four. But `candidate_partitions()` emits no MIA-style candidates, so it would work for `.splitter = "native"` only until the FIT/DLi/DGi kernels are taught to enumerate NA-left and NA-right variants of each candidate. That is real work, and it doubles the candidate count.

Option 5 — refit every leaf from `tree$fitted`. Rather than only the leaves lacking a fit, refit all of them, so every leaf at least agrees with `predict()`. Cheap to write, costs one IGSCA fit per leaf, and leaves the inner nodes and the stored split criteria still referring to the growth partition.

Option 6 — carry the growth subset out through the trafo. This one needs no partykit internals: `.extree_node()` builds a node's info from whatever the trafo returned (minus `estfun`), and `doTrees()` owns that trafo, so returning `subset` would put the growth-time row set directly on the node for `attach_leaf_fits()` to use. That makes the fits agree with the criteria stored beside them — but then `predict()` is the thing that disagrees. Same trade as option 5, resolved the other way.

4.2.4 Recommendation

Option 1 as the default, with option 2 available as an explicit opt-in for users who want the missing rows kept. That combination is small, honest, and leaves no silent randomness. Options 3 and 4 are worth revisiting only if missing covariates become a common case for this method.

Nothing here is currently exercised: the shipped fixture has no missing covariate values.

4.3 **not applied** `bdiagFit()` `dimname` suffixes differ between its two branches

4.3.1 Current behaviour

The multigroup branch suffixes each block with the group **name**; the single-group replication branch suffixes with `1..n_blocks`. Measured on `BergamiBagozzi2000` with the grouping variable relabelled `female/male`:

```
pooled, .n_blocks = 2 : ceil_1,      cei2_1,      ... ceil_2,      cei2_2,      ...
MGA,      .id = "sex" : ceil_female, cei2_female, ... ceil_male, cei2_male, ...
identical(dimnames) : FALSE
DL = 0.4196715   DG = 0.05112998      <- both correct
```

`calculatedDG()/calculatedDL()` index positionally and never look at `dimnames`, so every number is right. This is a labelling defect only.

It is also nearly invisible in practice, which is why it survived: inside `partition_stat()` the grouping column's levels are literally 1 and 2, so the two schemes coincide exactly — and even `BergamiBagozzi2000$gender` is coded 1/2, so the existing test fixture coincides too. It shows only when a user calls `bdiagFit()` directly with a genuinely named grouping variable.

4.3.2 The options

Option A — leave it, document it. Defensible: the numbers are right, and group names are the more informative of the two labellings. Costs nothing.

Option B — add a `.block_names` argument. Defaults preserve today’s behaviour exactly (`names(Sigma_list)` for the multigroup branch, `seq_len(.n_blocks)` for the replication branch), so nothing existing changes; supplying it lets a caller align the pooled matrix with the MGA it is about to be compared against. Prototyped, and the resulting output is:

```
pooled, .block_names = names(mga) : ceil_female, cei2_female, ... ceil_male, ...
MGA                               : ceil_female, cei2_female, ... ceil_male, ...
identical(dimnames)               : TRUE
DL = 0.4196715   DG = 0.05112998   <- unchanged
default, no .block_names          : ceil_1, cei2_1, ... ceil_2, cei2_2
```

Option C — index-suffix both branches. Makes them agree by discarding the group names. Loses information and churns the `paste0(rownames(Sigma), "_1")` assertion in `test-helper_doTrees.R`.

Option D — name-suffix both branches. Not possible: the pooled object has no groups to take names from. This is the asymmetry that produced the inconsistency in the first place.

4.3.3 Why option B is worth more than tidier labels

It is the only one that makes a **conformability assertion** expressible. Today

```
stopifnot(identical(dimnames(Si_pool), dimnames(Si_mga)))
```

cannot be added to `ndt_dists()`, because the two disagree by construction on exactly the comparisons the function exists to make. With `.block_names` threaded through from `partition_stat()` (`.block_names = names(mga$fit)`) the names match whenever the comparison is valid, and the assertion becomes a real check. It would catch a wrong `.n_blocks`, a group whose model carries a different indicator set, a construct-versus-indicator VCV mix-up, and a group-order flip — none of which any current code path detects, because positional indexing silently returns a number for all of them.

That is the argument for doing it: not the labels, but the assertion the labels currently prevent.

4.4 **not applied** The numeric candidate grid is exhaustive on purpose (withdrawn B5)

What was proposed, and applied. The unordered-factor branch of `candidate_partitions()` refuses more than 11 levels because “1023 candidate partitions, each costing a two-group IGSCA fit”, while the numeric branch emitted one candidate per distinct value with no ceiling. On the fixture’s `noise_1` left unrounded, that is 941 candidates at `minbucket = 30` — roughly 7.4 minutes for one covariate at one node, and about an hour for a `maxdepth = 3` tree. `max_cuts = 20L` exists in `dev/igsca/tree_template/igsca_tree.R`, so this was written up as a port regression and `max_cuts` was restored.

Why that was wrong. The removal was deliberate. Commit 719be7a3 (2026-08-28) is titled “removed `max_cuts`”, and the roxygen it installed in place of the parameter states the reason:

The scan is exhaustive: every midpoint between consecutive distinct values of a numeric covariate, every one of the $K - 1$ cuts of an ordered factor, and all $2^{(K - 1)} - 1$ bipartitions of an unordered one. This matches what partykit’s own scan considers, but the non-native splitters pay a two-group IGSCA fit per candidate where partykit pays arithmetic, so their

cost at a node is proportional to the covariate's cardinality. Whether the grid should be binned first, and how, is left open.

So the exhaustiveness is a **property being preserved**, not an oversight: the non-native splitters must consider exactly the candidate set partykit's native scan considers, or a comparison between `.splitter` values is confounded by the two paths having seen different grids. The cost was known and accepted, and the binning question was explicitly deferred.

Compounding detail. By the time of this review that rationale had already been trimmed to an empty roxygen description by a later commit, and the documentation pass then overwrote the empty description — removing the last trace of the decision from the file. The review should nonetheless have run `git log -S"max_cuts"`, which surfaces the commit immediately, before asserting that the port had dropped it.

Current state. `max_cuts` is fully reverted from `igsca_tree_control()`, `argmax_split()`, `candidate_partitions()` and the tests. The rationale paragraph has been restored to `candidate_partitions()`'s roxygen, adapted to the port's observed-value convention, and now carries the commit hash and an explicit “do not reinstate without settling that question”. A test, “`candidate_partitions()` scans the numeric grid exhaustively”, pins the property so that a future reader meets the decision as an assertion rather than as an absence.

What remains open is the question 719be7a3 left open: whether to bin the grid, and how. The measurement above is the argument for doing something eventually; the equivalence with partykit's scan is the constraint any scheme has to respect. A binning rule applied to **both** the native and non-native paths would preserve that equivalence, where `max_cuts` — which the native path ignores — did not.

5 Checked and found correct

Recorded so these are not re-litigated later. Several were reported as suspect and discarded on verification.

- **The Šidák note in @param bonferroni is right, and subtly so.** At the point `.extree_node()` multiplies by k , `criteria["p.value", ]` is stored on the $\log(1 - p)$ scale — visible only from the later `p["p.value", ] <- -expm1(p["p.value", ])`, which maps s to $1 - e^s$. So multiplying the stored value by k yields exactly $1 - (1 - p)^k$. Read at the multiplication site alone it looks like plain Bonferroni. The `@param` text and the inversion in the `R_test` test are both correct as written.
- `ef[subset, ] <- h` cannot misalign or recycle: `processData()` **errors** on incomplete cases rather than dropping rows, and `subset` is always ascending.
- `goes_left/sub_j` alignment through `argmax_split()` → `partition_stat()` is correct for all three candidate branches.
- `return()` inside `tryCatch(expr = {...})` genuinely returns from `partition_stat()`, not merely from the `tryCatch`.
- `real_scalar()` rejects complex correctly: `is.numeric()` is `FALSE` on complex and short-circuits before `is.finite()` (which is `TRUE` on complex).
- `calculateFIT`, `calculateDG` and `calculateDL` all return scalars on multigroup input, so `vapply(..., numeric(1))` is safe.
- `bdiagFit()` block **ordering** cannot matter: the pooled blocks are identical copies, so both distances are invariant to group order.
- The ordered-factor counterexample sought in the review does not exist: `kidids_split()` applies `.bincode()` to the integer codes and `match(levs[i], levels(z))` is exactly that code, including when the node is missing an interior level.
- `csem_converged()`'s NA path is fail-safe (`isTRUE(NA)` is `FALSE`).

- `tidy(parameters = "all")` emits only `Indirect_effect` and `Total_effect` from the effects table — exactly the two `coef()` filters — so no term collision is possible.
- `warn_dead_splitter()`'s `<` is equivalent to `!=` given `n_fail_split <= n_split_scan` by construction.
- Every `@noRd` / `@export` placement, cross-checked against `NAMESPACE`.
- The `## Stirling number of the second kind S(K, n = 2)` comment is correct ($S(K, 2) = 2^{K-1} - 1$), and the GPL attribution above it must stay.
- `influence_mat()`'s "N x (J + P)" matches `calculateGSCAErrors()`.

The following were considered for trimming and deliberately left at full length, because each records knowledge that cost a source dive: the Šidák note; the `modifyList()` warning and the `$<--` coerces note in `fit_csem()`; the complex-eigenvalue comment in `ndt_dists()`; `@param minprob`'s "the proportion is of the **node**, not of the sample"; `multiway_split()`'s transcription of undocumented partykit behaviour; the three `stopifnot()` messages ending "partykit has changed since initial doTrees development"; and the `splitfun-contract` comment in the tests.

6 Tests added

Test	Pins
a root fit failure is booked once, not again as a leaf failure	B2
<code>root_criteria()</code> is NULL whenever no test ran, not only on failure	B1
<code>coef()</code> keeps its matrix shape when every leaf refit failed	B7
<code>candidate_partitions()</code> scans the numeric grid exhaustively	4.4
<code>validate_tree_input()</code> refuses an unaffordable factor by name, up front	B8

B3 and B6 are covered by existing tests that continue to pass under the new semantics; B4 is covered by the existing `verify` file.